

심장병 예측 분석 모델

SCESecure (김기원)

경고 : 해당 모델은 물론 사용된 데이터 셋은 의사결정을 보조하는 수단이며, 절대 단독으로 결정하는 시스템은 아닙니다.

1. 문제 정의와 사용 목적

- 이 모델은 UCI에서 제공하는 UCI Heart Disease(CardioCare) 데이터 셋을 사용한 모델입니다. 데이터 셋에서는 특정 타겟의 나이, 성별, 흉통, 혈압(휴식기), 콜레스테롤, 공복혈당(FBS), 안정시 심전도(Resting ECG), 최대 혈압 수치, 운동협심증, ST 분절 수치, 심혈관 직경에 따른 심장병 등에 대한 데이터를 가지고 있습니다. (더 자세한 건 <https://archive.ics.uci.edu/dataset/45/heart+disease> 확인 바랍니다.)
- 이 데이터 셋의 특성 중 5개의 특성(나이, 혈압(휴식기), 콜레스테롤, 최대 혈압 수치, ST 분절 하강 수치)이 훈련 데이터로, 심장병 데이터를 테스트 데이터로 하여 사용되었습니다. 따라서 타겟 환자의 훈련 데이터에 사용된 5개의 특성들을 모델에 입력하면 심장병 유무를 판별할 수 있도록 하는 것이 목적입니다.

2. EDA 핵심 결과

- 심장병 데이터의 경우 아래와 같이 이진화 하였습니다.

```
target_data['binary_num'] = pd.cut(target_data['num'],
                                   bins=[-np.inf, 0, np.inf],
                                   labels=[0, 1])
print(target_data)
```

[37]

...	Unnamed: 0	num	binary_num
0	0	0	0
1	1	2	1
2	2	1	1
3	3	0	0
4	4	0	0
..	...	...	...
298	298	1	1
299	299	2	1
300	300	3	1
301	301	1	1
302	302	0	0

[303 rows x 3 columns]

- 결측치를 제거하고, 이상치의 경우 StandardScaler를 통하여 조정하는 파이프라인을 구축한 결과 아래와 같이 되었습니다.

```
def preprocessing(data) :
    data.dropna() # 빈 column 제거
    data.drop_duplicates() # 중복 특성 제거

    # 파이프라인 (결측치 제거 -> 이상치를 Standardization으로 하여 조정)
    pipeline = Pipeline([('NA_Processing', KNNImputer(n_neighbors=5)),
                          | ('Scaler_Processing', StandardScaler())])

    preprocessed_data = pipeline.fit_transform(data) # 파이프라인 적용

    return preprocessed_data
```

3. EDA 결과에 근거한 전처리 결정

- 우선 모든 데이터들을 대상으로 하여 구조와 타입들을 확인하였습니다. 총 303개의 데이터를 가지고 있었고, 타입의 경우 oldpeak, ca, thal 특성은 float64, 나머지는 모두 int64이었습니다. 이 부분의 경우 때에 따라 타입을 바꾸는 방향으로 진행하겠습니다.
- 심장병 데이터에 대하여 클래스 분포를 확인해본 결과 그 정도에 따라서 0, 1, 2 등으로 나뉘는 것이 확인되었습니다. 따라서 이진화가 필요하였습니다.
- 결측치를 확인해보니 모든 데이터에 존재하였습니다. 따라서 NaN 과 같은 결측치를 제거 할 필요가 있었습니다.
- 이상치의 경우 연속적인 특성들에 대하여 boxplot을 사용하여 확인해본 결과 chol, trestbps, thalach, oldpeak 특성에 주로 존재하였습니다. 따라서 해당 값의 분포를 조정할 필요가 있었습니다. (따라서 이를 이용한 것이 StandardScaler 입니다.)

4. MLflow 실험 기반 모델 비교 표 및 최종 선택 정당화.

- 모델은 총 5개의 모델(Logistic Regression, SVM(Support Vector Machine), Random Forest, KNN, XGBoost)이 훈련 및 테스트되어 사용되었습니다. 이들에 대하여 MLflow 실험한 결과 다음과 같은 결과가 나타났습니다. (사용된 지표는 아래 표를 확인 바라며, 반올림하여 소수점 둘째자리까지 나타냈습니다.)

	Logistic Regression	SVM	Random Forest	KNN	XGBoost
정확도 (Accuracy)	0.55	0.57	0.58	0.56	0.49
균형 정확도 (Balanced Accuracy)	0.27	0.30	0.35	0.29	0.27
ROC AUC (one-vs-one)	0.64	0.53	0.64	0.68	0.61
ROC AUC (one-vs-rest)	0.73	0.57	0.70	0.71	0.65
정밀도 (Precision)	0.65	0.61	0.69	0.65	0.70
민감도 (Recall)	0.94	0.96	0.90	0.92	0.80
F1 점수 (F1 Score)	0.77	0.75	0.78	0.76	0.74

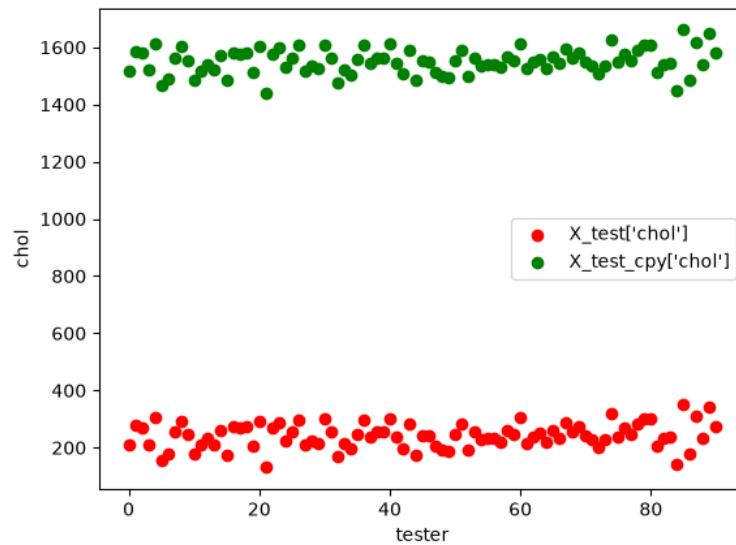
- 위 표를 확인해보면 지표에 따라 대부분 최적의 모델이 제각각이라는 점을 알 수 있습니다. 그나마 Random Forest 모델이 정확도와 F1 점수 모두 높다는 점을 알 수 있지만, 상황에 따라서 지표를 선택해야 합니다.
해당 모델을 사용처는 병원에서 사용되며, 목적은 위에서 이미 언급했습니다. 따라서 가장 중요한 지표는 바로 민감도(Recall)입니다.
그래서 민감도가 0.96으로 가장 높은 모델은 SVM으로 이를 선택하여 진행합니다.
- SVM 모델에 한하여 GridSearchCV 방법으로 하이퍼파라미터 튜닝하였고, 결론적으로 튜닝한 파라미터에 대하여 SVM 모델에 적용하여 테스트와 모니터링에 해당 모델을 적용하였습니다.
- (하이퍼파라미터 튜닝을 한 모델은 SVM 뿐만이 아닙니다. Random Forest와 KNN 모델에 대해서 또한 튜닝을 하였는데 각각 정확도 및 F1 점수와 ROC AUC 수치가 높았기 때문에 혹시 몰라 마찬가지로 튜닝하였습니다. 일반적으로 해당 지표들 또한 중요한 쪽에 속하기 때문입니다.)

5. 테스트와 패키징

- 테스트 할 때, 대상이 되는 모델은 위에서 이미 언급하였습니다. 따라서 이 섹션에서는 해당 모델에 대해서 추가 설명하지 않겠습니다.
- 테스트는 unittest를 기본으로 하여 테스트되며, 모델에 학습될 데이터는 특성 테스트 데이터(이하 X_test 데이터)를 사용하였습니다.
- 테스트는 3가지의 테스트(데이터 셋의 shape, 예측 확률, 입력값 범위 검증)를 진행하였고 모델들을 모두 무사히 통과합니다.
- 위 3가지 테스트 중 입력값 범위 검증에 대한 테스트에서는 나이와 콜레스테롤을 중점으로 검증하였는데, 그 이유는 해당 특성들이 임상적으로 범위가 정해져 있기 때문입니다. 이에 따라 해당 특성들을 선택하였습니다.
- 패키징에 대해서 우선, 해당 프로젝트는 python 3.13을 기본으로 돌아갑니다. 또한 이름은 'cardiocare'라는 Working 디렉토리가 생성 및 저장됩니다. 기본적으로 requirements.txt 파일을 통해 프로젝트에 사용되는 패키지들을 설치하고 나면, data/ , src/ , tests/ 폴더 내에 있는 파일들을 복사합니다.
엔트리 포인트는 tests/ 폴더 내에 있는 test_pipeline.py입니다.
(해당 Docker 패키징에 관해서는 Dockerfile 파일 내에 과정이 담겨져 있습니다.)

6. 드리프트 결과 및 재학습 / 피드백 루프 계획

- 데이터 드리프트 현상을 재현 하기 위해 X_test 데이터 셋을 복제한 후(이하 X_test_cpy), X_test_cpy에서 chol 특성의 평균을 이동시켰습니다. (chol 특성의 표준편차에 30을 곱한 다음 해당 값을 더하는 방식입니다.)
- 이렇게 되면 복제된 해당 데이터 셋은 데이터 드리프트 현상을 충분히 재현하였습니다. (Kolmogorov-Smirnov 검증 해본 결과 p_value가 5.52411833e-54 으로 나타났습니다. 자세한 건 monitor.py 확인 바랍니다.)
- 이를 토대로 모델에 적용하여 학습시킨 결과, 균형 정확도 값의 경우 X_test가 0.22222222222222224, X_test_cpy는 0.2 가 출력되었습니다.
- 참고 : 두 데이터 셋의 chol 특성에 대한 데이터들은 다음과 같이 나타났습니다.



- 만일 해당 모델에서 위와 같이 데이터 드리프트가 나타난다면, 이에 대하여 모델을 재훈련을 할 필요가 있습니다.
- 해당 문제는 데이터에 관련한 문제이기에 현재 새로운 데이터와 이전 데이터를 결합한 다음, 이를 그대로 모델에 학습만 시키면 됩니다.

7. 서빙 방식

- 모델을 서빙할 때는 Model-as-a-Service과 On-Device 중 하나를 골라야 하는데, On-Device를 골라야 한다고 봅니다. 해당 모델은 임상적인 정보(즉, 이는 개인정보에 해당합니다.)가 존재하기에 이에 대하여 다루기 위해선 On-Device가 적합하다고 생각합니다.
- 또한 해당 모델은 병원에서 주로 사용될 것이기에, 빠르게 반응 및 결론을 내리기 위해서는 On-Device가 적합하다고 생각합니다.

8. 한계, 윤리적 고려 사항 (v1.0 기준)

- 해당 모델에는 물론 몇 가지 한계점이 존재합니다. 우선 가장 큰 문제로 정확도가 객관적으로 보면 매우 떨어집니다.
(monitor.py를 실행해보면 자세히 알 수 있겠지만, 전체 91개의 데이터 중 50개에 대하여 예측 성공하였습니다. (약 54.95%))
- 그리고 시간 측면에서는 매우 비효율적입니다. 현재 프로젝트에서 모델 및 파라미터를 따로 저장하거나 불러오는 부분이 없다 보니, 테스트나 모니터링 할 때

처음 전처리와 훈련하고 하이퍼파라미터 튜닝까지 전 과정을 거쳐야, 해당 코드를 실행할 수 있습니다.

- 내부 코드가 매우 비효율적으로 짜여져 있다보니, 가독성이 매우 어렵습니다.
- 윤리적인 측면에서도 문제가 있습니다. 만일 이 모델을 실제 병원에서 심장병을 예측하는 데에 사용할 경우 환자에게 정확하게 판단을 내리지 못할 수 있습니다. (맨 앞 페이지에서 언급했듯이 **이 모델과 데이터 셋 모두 심장병 전문의의 의사 결정을 도와주는, 즉 보조하는 수단이며, 절대 단독으로 결정하는 수단이 아닙니다!**)
- 만일 여기서 1주라는 시간이 더 주어진다면, 아마 위의 문제들 중 시간 문제, 코드 문제 등을 잘 해결할 수 있을 것 같습니다.
(정확도 문제는 실력이 부족하여 나타난 것 같고, 윤리적인 문제는 해당 모델의 특성 상 나타나는 것 같습니다. 따라서 정확도 문제는 시간이 더 걸릴 것 같고, 윤리적인 문제는 사회가 뒷받침해줘야 해결이 가능할 것 같습니다.)

9. AI 사용 여부

- 이 프로젝트를 사용할 때 AI(Anthropic 사의 Claude)의 도움을 일부 받았습니다. 또한 AI 사용 전에 가능하면 인터넷에 검색을 하여 해결할 수 있도록 노력하였습니다. (인터넷을 검색을 하여도 참고 정도로 하였습니다.)
- 일반적으로 디버깅과 모르는 개념들, 아무리 검색해도 구현이 안되는 부분들에 한해서 사용하였고, 사용한 프롬프트 및 내역들은 아래와 같습니다.
 - <https://claude.ai/share/c156a613-253e-4ad6-a710-a77d9081a4f3>
 - <https://claude.ai/share/2c860fa5-7919-4261-bd65-a1b276c3ea66>
 - <https://claude.ai/share/97f74b25-55fa-466e-80ff-514ae84945b4>
 - <https://claude.ai/share/f8da91c8-e443-4187-b872-70542c5be905>
 - <https://claude.ai/share/609c9b38-ff19-4dfa-8c47-9c61154489b4>
 - <https://claude.ai/share/70d51345-ced3-4a1a-a87d-e3e54029d121>
 - <https://claude.ai/share/0682ec96-57db-457f-a4a2-24b7102e7196>
 - <https://claude.ai/share/e571b18c-a76b-4e85-8118-2f530693a239>
 - <https://claude.ai/share/a1da8d86-458a-48c2-81fe-83ec05f11a4b>
 - <https://claude.ai/share/ba66760e-cada-4b18-b152-03b95baba284>
 - <https://claude.ai/share/fa70192a-6998-440a-838e-1c3d09c51d19>
 - <https://claude.ai/share/ee68b7ec-973c-4b15-b389-2e41df2eea24>
 - <https://claude.ai/share/095f9391-3106-4ca5-afc7-a4169d57c33a>